

**Министерство науки и высшего образования Российской
Федерации ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ
АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО**

**Лабораторная работа №2
по дисциплине
“Фронт-энд разработка”**

Семестр IV

**Выполнил:
студент
Ковров Евгений Вадимович
гр. J3212
ИСУ 466206**

**Санкт-Петербург
2026**

1 Цель работы

Целью лабораторной работы является получение практических навыков взаимодействия клиентского веб-приложения с внешним API. В рамках работы необходимо реализовать моковое API с использованием `json-server`, подключить его к фронтенду и обеспечить базовые сценарии работы приложения: загрузку данных, авторизацию и выполнение пользовательских действий через API.

2 Задача

В рамках лабораторной работы требовалось:

- Развернуть моковое API с использованием `json-server`
- Организовать хранение данных (пользователи, мероприятия, заказы)
- Реализовать получение списка мероприятий с API
- Реализовать страницу деталей мероприятия с загрузкой по `id`
- Добавить авторизацию (регистрация и вход)
- Реализовать процесс покупки билета через API
- Реализовать страницу «Мои билеты» с заказами пользователя

3 Ход работы

3.1 Настройка mock API

Для имитации серверной части использован `json-server`. Создан файл `db.json`, содержащий коллекции:

- `users`
- `events`
- `orders`

API запускается командой:

```
npm install
npm run server
```

После запуска API доступен по адресу: `http://localhost:3000`

```
{
  {
    "id": "1",
    "title": "Джаз в Полуторке",
    "description": "Камерный вечер джаза в Новой Голландии с живым составом и мягким сценическим светом. В программе стандарты и современные аранжировки, рассчитанные на спокойный вечер в центре Петербурга.",
    "date": "2026-04-10",
    "time": "20:00",
    "city": "Санкт-Петербург",
    "venue": "Новая Голландия, сцена «Полуторка»",
    "price": 1200,
    "image": "assets/img/events/jazz.jpg",
    "category": "Концерт",
    "availableTickets": 44
  },
  {
    "id": "2",
    "title": "Стендап: Пятничный микрофон",
    "description": "Пятничный стендап-вечер с короткими сетами, импровизацией и плотным лайнапом. Формат подойдёт тем, кто любит камерные площадки, быстрый темп и живую реакцию зала.",
    "date": "2026-04-12",
    "time": "19:30",
    "city": "Санкт-Петербург",
    "venue": "Бар на Рубинштейна",
    "price": 700,
    "image": "assets/img/events/standup.jpg",
    "category": "Стендап",
    "availableTickets": 31
  },
}
```

Рис. 1: Ответ API при запросе списка мероприятий

3.2 Подключение API на клиенте

Создан общий axios-клиент с базовым URL:

```
http://localhost:3000
```

Реализован сервис работы с мероприятиями:

- получение списка мероприятий
- получение мероприятия по id

Данные загружаются асинхронно и отображаются в интерфейсе.

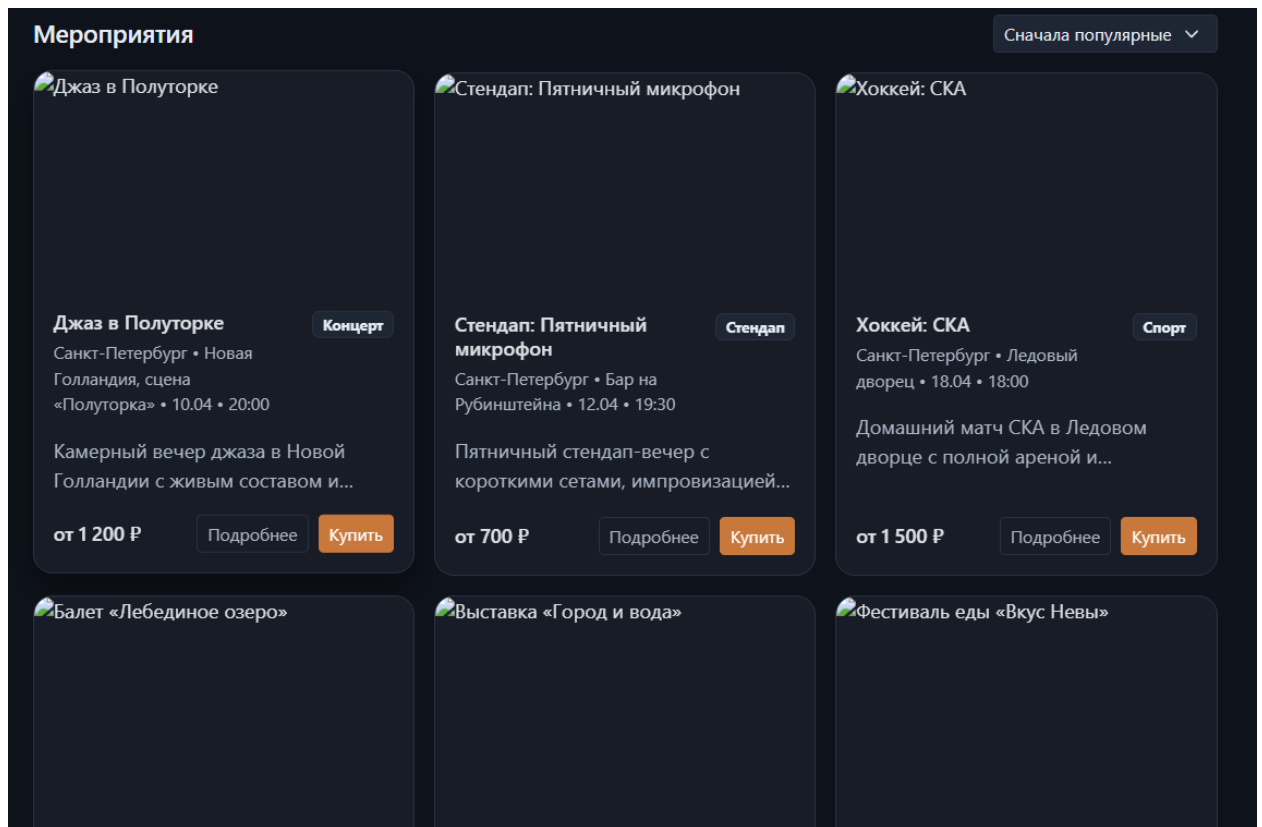


Рис. 2: Каталог мероприятий, загруженный с API

3.3 Страница мероприятия

Создана единая страница `event.html`, которая:

- получает `id` из `query`-параметра
- запрашивает данные через API
- отображает информацию о мероприятии

Реализованы состояния:

- загрузка
- ошибка
- мероприятие не найдено

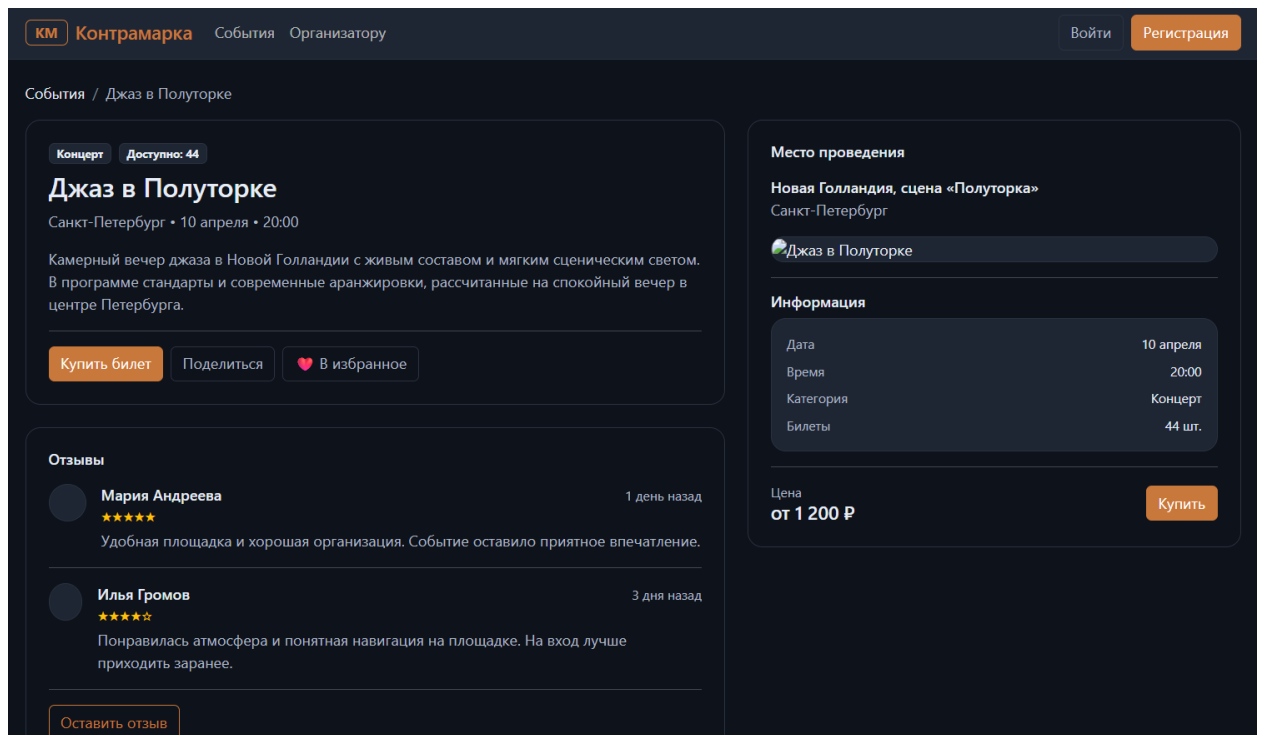


Рис. 3: Страница мероприятия

3.4 Авторизация

Реализован mock auth через API:

- регистрация пользователя (POST /users)
- вход (GET /users?email=&password=)

Текущий пользователь сохраняется в `localStorage`.

Добавлены страницы:

- `login.html`
- `register.html`

Также реализовано отображение состояния авторизации в навигации.

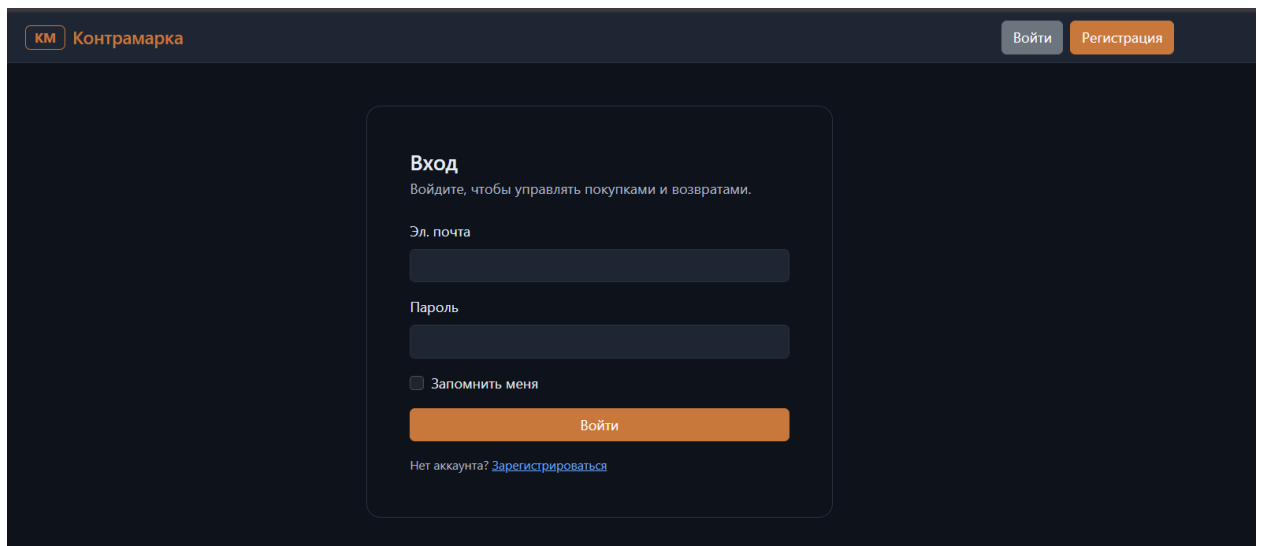


Рис. 4: Страница входа

3.5 Покупка билетов

Реализован процесс покупки:

- если пользователь не авторизован — редирект на login
- после входа — возврат обратно
- создаётся заказ через `POST /orders`
- уменьшается количество доступных билетов через `PATCH /events`

После успешной покупки интерфейс обновляется без перезагрузки страницы.

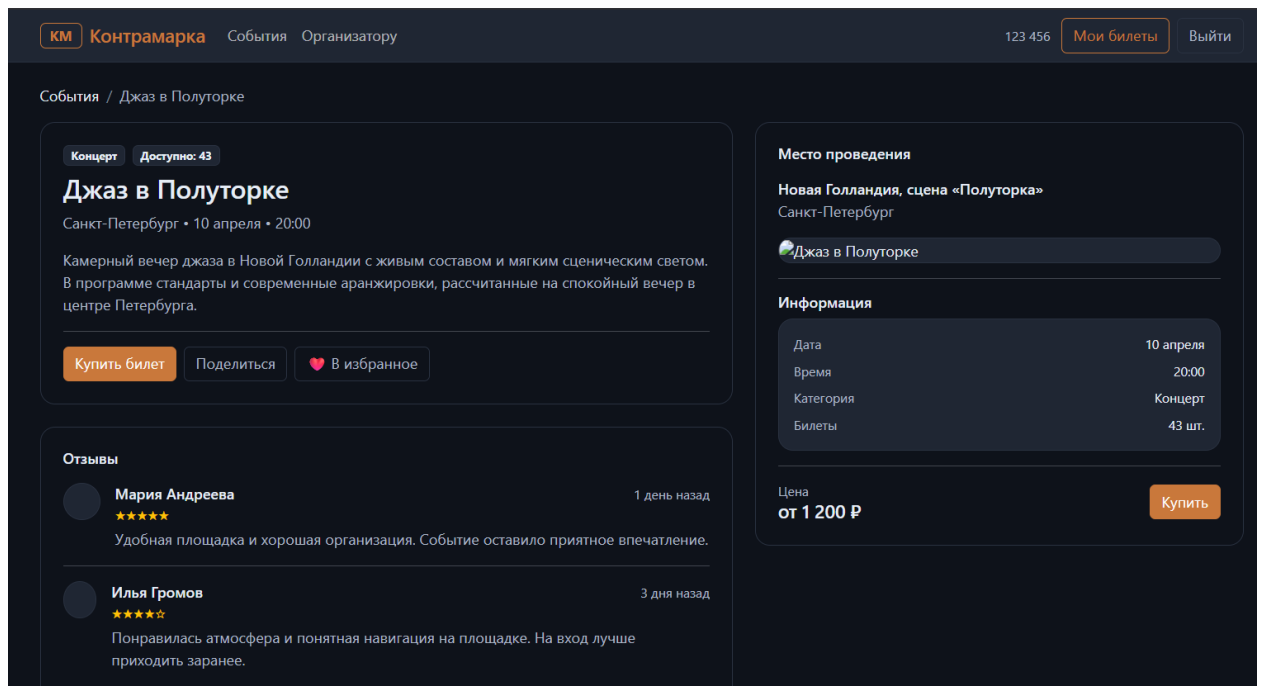


Рис. 5: Процесс покупки билета

3.6 Страница заказов

Создана страница `orders.html`:

- доступна только авторизованным пользователям
- загружает заказы пользователя через API
- отображает информацию о каждом заказе

Для каждого заказа дополнительно подгружается информация о мероприятии.

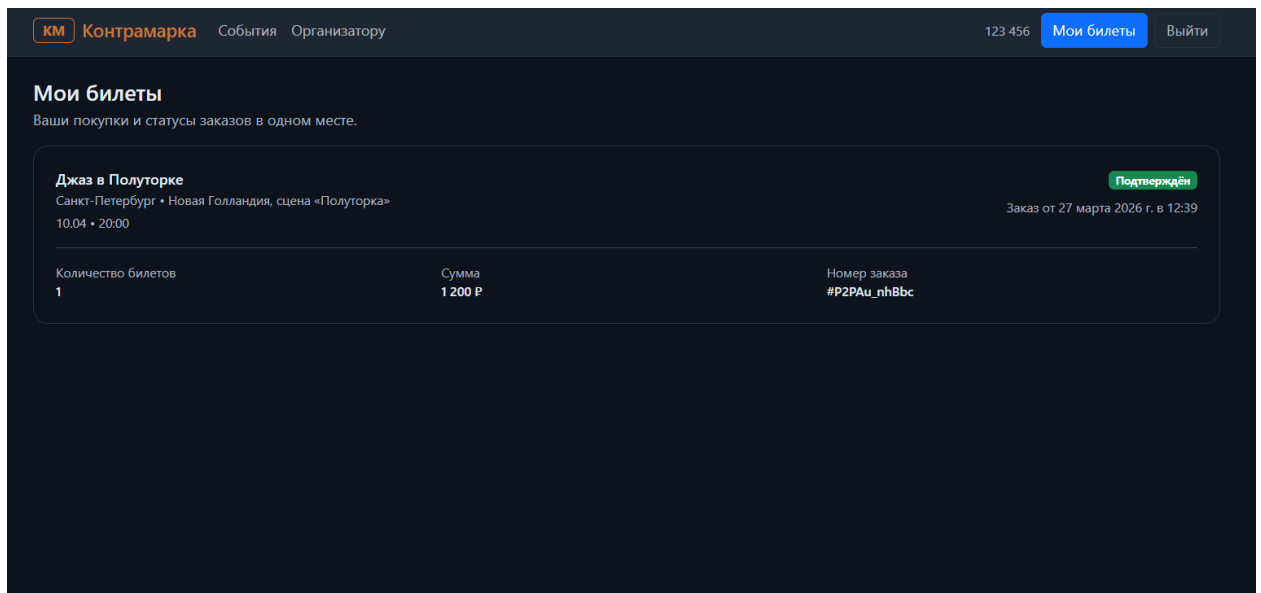


Рис. 6: Страница «Мои билеты»

3.7 Обработка ошибок

Во всех частях приложения реализована обработка ошибок:

- сетевые ошибки
- недоступность API
- некорректные данные

Пользователю отображаются понятные сообщения, без технических деталей.

4 Вывод

В ходе лабораторной работы было реализовано взаимодействие фронтенд-приложения с внешним API на основе `json-server`.

Приложение было доработано до состояния, в котором все основные пользовательские сценарии (просмотр мероприятий, авторизация, покупка билетов, просмотр заказов) выполняются через API.

Получен практический опыт работы с асинхронными запросами, организацией клиентских сервисов и обработкой ошибок.